

TD7 - Les curseurs PL/pgSQL (partie 2)

Matisse

Kra

BTS SIO

Sommaire

Sommaire	1
Introduction	2
Exercice 1 : Nombre de lignes modifiées et SELECT INTO	2
Exercice 2 : Attributs d'un curseur	4
Exercice 3 : Erreurs courantes sur les curseurs	5
Exercice 4 : Parcourir un curseur avec une boucle	7
Exercice 5 : Curseur paramétré et boucle imbriquée	9
Exercice 6 : Verrouiller et modifier avec FOR UPDATE	11
Exercice 7 : Mini-projet, rapport de paie par département	14
Compte rendu final	16
Conclusion	17

Introduction

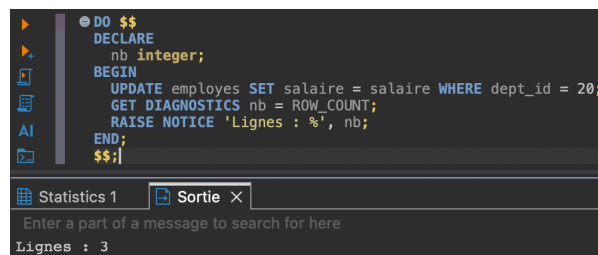
Ce TD prolonge l'étude des curseurs en PL/pgSQL, le langage procédural de PostgreSQL. Un curseur permet de parcourir ligne par ligne le résultat d'une requête qui en renvoie plusieurs, là où SELECT INTO ne sait en traiter qu'une seule. Dans cette seconde partie, je travaille sur deux tables, departements et employes, sous DBeaver connecté à PostgreSQL.

Les énoncés sont écrits pour Oracle, ce qui m'a obligé à adapter plusieurs notions. DBMS_OUTPUT.PUT_LINE devient RAISE NOTICE, SQL%ROWCOUNT devient GET DIAGNOSTICS ... = ROW_COUNT, les blocs anonymes s'écrivent DO \$\$... \$\$, et les attributs %ISOPEN, %FOUND, %NOTFOUND et %ROWCOUNT n'existent pas tels quels dans PostgreSQL. Les sept exercices sont présentés ci-dessous, chacun avec le code écrit, son principe de fonctionnement et les captures des résultats, puis viennent les réponses aux trois questions du compte rendu final.

Exercice 1 : Nombre de lignes modifiées et SELECT INTO

Dans la première partie, j'ai écrit un bloc DO qui exécute un UPDATE sur le département 20, puis lit le nombre de lignes touchées. L'UPDATE affecte salaire = salaire à chaque ligne, ce qui ne modifie aucune valeur mais permet de compter les lignes concernées. Là où Oracle utilise SQL%ROWCOUNT, PostgreSQL demande de récupérer ce nombre avec GET DIAGNOSTICS dans une variable entière, que j'ai ensuite affichée avec RAISE NOTICE. L'onglet Sortie de DBeaver affiche « Lignes : 3 », ce qui correspond aux trois employés du département 20.

```
GET DIAGNOSTICS nb = ROW_COUNT;
```



```
DO $$
DECLARE
  nb integer;
BEGIN
  UPDATE employes SET salaire = salaire WHERE dept_id = 20;
  GET DIAGNOSTICS nb = ROW_COUNT;
  RAISE NOTICE 'Lignes : %', nb;
END;
$$;
```

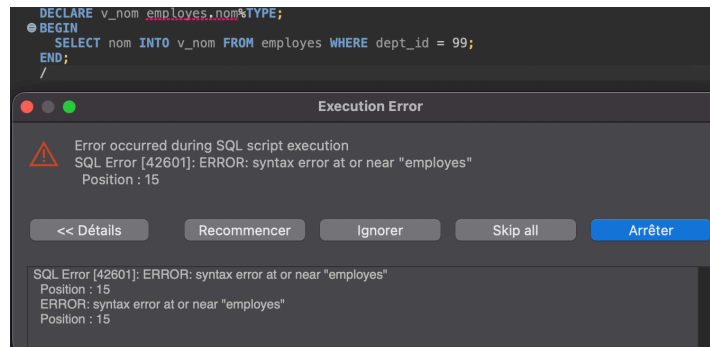
Statistics 1 | Sortie X

Enter a part of a message to search for here

Lignes : 3

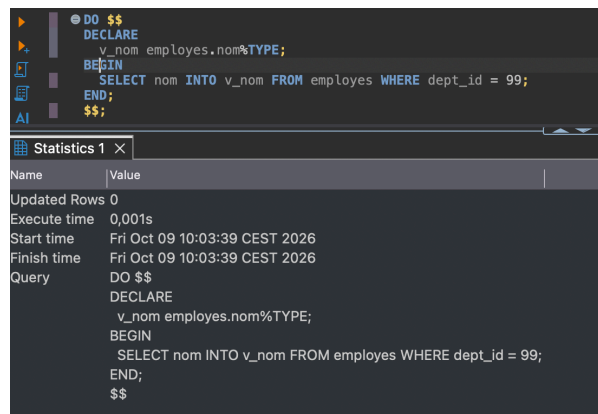
Nombre de lignes modifiées récupéré avec GET DIAGNOSTICS

La seconde partie portait sur SELECT INTO. J'ai déclaré une variable v_nom avec le type employes.nom%TYPE, qui reprend automatiquement le type de la colonne. Ma première exécution a échoué avec l'erreur de syntaxe 42601 en position 15, car j'avais écrit le bloc DECLARE ... BEGIN ... END; sans l'envelopper dans DO \$\$... \$. PostgreSQL a alors lu DECLARE comme la commande qui déclare un curseur et s'est arrêté sur le mot employes. Une fois le bloc enveloppé dans DO \$\$, il s'est exécuté.

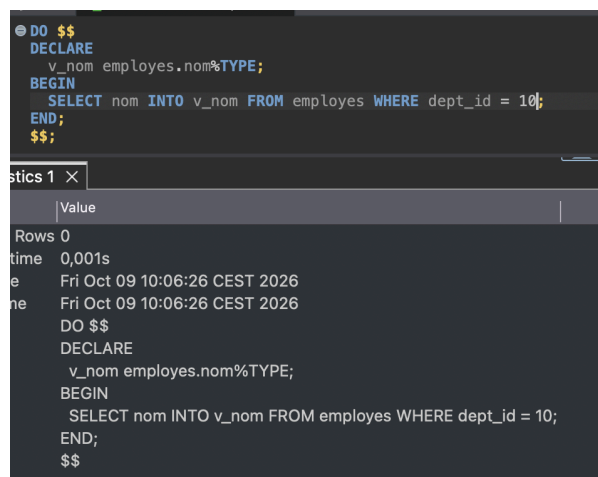


Erreur 42601 : bloc exécuté sans l'enveloppe DO \$\$

J'ai ensuite testé `SELECT nom INTO v_nom` avec le département 99, qui ne contient aucun employé, puis avec le département 10, qui en contient plusieurs. Dans les deux cas, DBeaver n'affiche aucune erreur : sans `STRICT`, PostgreSQL affecte la première ligne trouvée et ignore les suivantes, et ne signale rien lorsque la requête ne renvoie aucune ligne. Ce comportement est silencieux, ce qui peut cacher des erreurs dans les données.



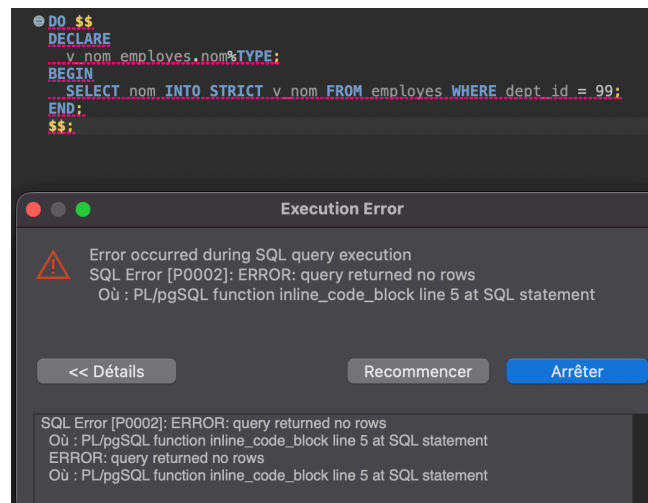
SELECT INTO sur le département 99 : aucune ligne, aucune erreur



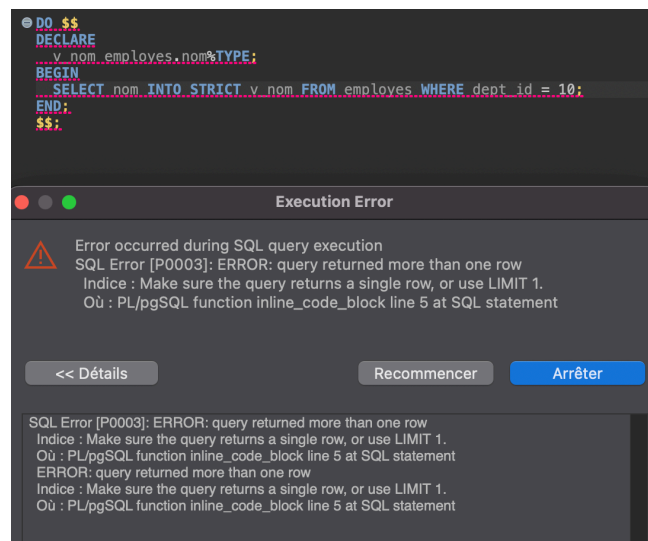
SELECT INTO sur le département 10 : plusieurs lignes, aucune erreur

Avec `INTO STRICT`, les deux cas deviennent des erreurs. Le département 99 provoque l'erreur P0002 « query returned no rows », et le département 10 provoque l'erreur P0003 « query returned more than one row », accompagnée de l'indice de PostgreSQL qui conseille de s'assurer que la requête renvoie

une seule ligne ou d'utiliser LIMIT 1. STRICT impose donc que la requête renvoie exactement une ligne.



INTO STRICT sur le département 99 : erreur P0002



INTO STRICT sur le département 10 : erreur P0003

Exercice 2 : Attributs d'un curseur

Cet exercice demandait d'afficher l'état d'un curseur avant le OPEN, après le OPEN, après un FETCH et après le CLOSE. J'ai déclaré un curseur curs1 qui lit le nom et le salaire des employés du département 30, ainsi que deux variables typées avec %TYPE pour recevoir les valeurs. PostgreSQL n'a pas d'attributs %ISOPEN ni %ROWCOUNT sur les curseurs, j'ai donc remplacé chacun à ma façon. Pour savoir si le curseur est ouvert, je cherche son nom dans la vue système pg_cursors, en utilisant curs1::text car PostgreSQL le nomme en interne « <unnamed portal N> ». Pour ROWCOUNT, j'utilise un compteur nb_lignes que j'incrmente après un FETCH seulement si FOUND est vrai. FOUND est en revanche une vraie variable de PL/pgSQL, tout comme NOT FOUND.

```
est_ouvert := EXISTS (SELECT 1 FROM pg_cursors WHERE name = curs1::text);
```

```
DO $$
DECLARE
    est_ouvert boolean;
    nb_lignes integer := 0;
    nom_employees employees.nom%TYPE;
    salaire_employees employees.salaire%TYPE;
    curs1 CURSOR FOR SELECT nom, salaire FROM employees WHERE dept_id = 30;
BEGIN
    est_ouvert := EXISTS (SELECT 1 FROM pg_cursors WHERE name = curs1::text);
    RAISE NOTICE 'Avant OPEN : ISOPEN=%, FOUND=%, NOTFOUND=%, ROWCOUNT=%',
        est_ouvert, FOUND, NOT FOUND, nb_lignes;

    OPEN curs1;
    est_ouvert := EXISTS (SELECT 1 FROM pg_cursors WHERE name = curs1::text);
    RAISE NOTICE 'Après OPEN : ISOPEN=%, FOUND=%, NOTFOUND=%, ROWCOUNT=%',
        est_ouvert, FOUND, NOT FOUND, nb_lignes;

    FETCH NEXT FROM curs1 INTO nom_employees, salaire_employees;
    IF FOUND THEN nb_lignes := nb_lignes + 1; END IF;
    est_ouvert := EXISTS (SELECT 1 FROM pg_cursors WHERE name = curs1::text);
    RAISE NOTICE 'Après FETCH : ISOPEN=%, FOUND=%, NOTFOUND=%, ROWCOUNT=%',
        est_ouvert, FOUND, NOT FOUND, nb_lignes;
    RAISE NOTICE 'Nom : %, salaire : %', nom_employees, salaire_employees;

    CLOSE curs1;
    est_ouvert := EXISTS (SELECT 1 FROM pg_cursors WHERE name = curs1::text);
    RAISE NOTICE 'Après CLOSE : ISOPEN=%, FOUND=%, NOTFOUND=%, ROWCOUNT=%',
        est_ouvert, FOUND, NOT FOUND, nb_lignes;
END;
$$;
```

Bloc affichant ISOPEN, FOUND, NOTFOUND et ROWCOUNT à chaque étape

Avant l'exécution, j'avais prédit qu'après le OPEN, ISOPEN serait vrai, FOUND vrai, NOTFOUND faux et ROWCOUNT égal à 1. Le résultat est différent : après le OPEN, ISOPEN est vrai mais FOUND reste faux, NOTFOUND reste vrai et le compteur reste à 0. OPEN exécute seulement la requête, il ne lit aucune ligne, et FOUND ne change donc qu'au premier FETCH. Après le FETCH, les valeurs sont ISOPEN vrai, FOUND vrai, NOTFOUND faux et ROWCOUNT 1, avec la ligne Garcia à 4100. Après le CLOSE, ISOPEN repasse à faux, mais FOUND reste vrai : il garde la valeur du dernier FETCH et n'est pas remis à zéro par la fermeture.

```
Avant OPEN : ISOPEN=f, FOUND=f, NOTFOUND=t, ROWCOUNT=0
Après OPEN : ISOPEN=t, FOUND=f, NOTFOUND=t, ROWCOUNT=0
Après FETCH : ISOPEN=t, FOUND=t, NOTFOUND=f, ROWCOUNT=1
Nom : Garcia, salaire : 4100.00
Après CLOSE : ISOPEN=f, FOUND=t, NOTFOUND=f, ROWCOUNT=1
```

État du curseur avant OPEN, après OPEN, après FETCH et après CLOSE

Exercice 3 : Erreurs courantes sur les curseurs

Cet exercice consistait à provoquer volontairement les erreurs classiques sur un curseur et à noter chacune d'elles. J'ai d'abord essayé un FETCH avant le OPEN : PostgreSQL répond par l'erreur 22004 « cursor variable curs1 is null », car le curseur n'a pas encore été ouvert.

```
SQL Error [22004]: ERROR: cursor variable "curs1" is null
Où : PL/pgSQL function inline_code_block line 7 at FETCH
...curs1 CURSOR FOR SELECT nom, salaire FROM employees WHERE dept_id = 30;
...BEGIN
...FETCH NEXT FROM curs1 INTO nom_employees, salaire_employees;
...END;
...$;$;
```

FETCH avant OPEN : erreur 22004

J'ai ensuite fait un FETCH dans des variables de mauvais type. L'erreur 22P02 « invalid input syntax for type integer: "Garcia" » apparaît, car PostgreSQL essaie de convertir le nom de l'employé en entier.

```

DO $$
SQL Error [22P02]: ERROR: invalid input syntax for type integer: "Garcia"
Où : PL/pgSQL function inline_code_block line 8 at FETCH
curs1 CURSOR FOR SELECT nom, salaire FROM employees WHERE dept_id = 30;
BEGIN
OPEN curs1;
FETCH NEXT FROM curs1 INTO nom_employees, salaire_employees;
END;
$$;

```

FETCH dans une variable de mauvais type : erreur 22P02

Un double OPEN sur le même curseur produit l'erreur 42P03 « cursor "<unnamed portal 7>" already in use », et un double CLOSE produit l'erreur 34000 « cursor "<unnamed portal 8>" does not exist ». Dans les deux cas, le message indique la ligne du bloc concernée, ce qui facilite le diagnostic.

```

DO $$
SQL Error [42P03]: ERROR: cursor "<unnamed portal 7>" already in use
Où : PL/pgSQL function inline_code_block line 8 at OPEN
curs1 CURSOR FOR SELECT nom, salaire FROM employees WHERE dept_id = 30;
BEGIN
OPEN curs1;
OPEN curs1;
END;
$$;

```

Deux OPEN de suite : erreur 42P03

```

DO $$
SQL Error [34000]: ERROR: cursor "<unnamed portal 8>" does not exist
Où : PL/pgSQL function inline_code_block line 9 at CLOSE
curs1 CURSOR FOR SELECT nom, salaire FROM employees WHERE dept_id = 30;
BEGIN
OPEN curs1;
CLOSE curs1;
CLOSE curs1;
END;
$$;

```

Deux CLOSE de suite : erreur 34000

J'ai aussi essayé un FETCH avec une seule variable pour un curseur qui renvoie deux colonnes (nom et salaire). PostgreSQL n'a signalé aucune erreur et la variable a bien reçu le nom Garcia, ce qui montre que cette erreur-là n'en est pas une avec une variable scalaire.

```

DO $$
DECLARE
nom_employees employees.nom%TYPE;
curs1 CURSOR FOR SELECT nom, salaire FROM employees WHERE dept_id = 30;
BEGIN
OPEN curs1;
FETCH NEXT FROM curs1 INTO nom_employees;
RAISE NOTICE 'Nom : %', nom_employees;
CLOSE curs1;
END;
$$;

```

Statistics 1 | Statistics 2 | Statistics 3 | Sortie x

Enter a part of a message to search for here

: Garcia

FETCH avec une seule variable pour deux colonnes : pas d'erreur

Pour le défi, j'ai intercepté les deux erreurs dans le même bloc au lieu de laisser le bloc s'arrêter. J'ai entouré le second OPEN d'un sous-bloc avec EXCEPTION WHEN duplicate_cursor, et le second CLOSE d'un sous-bloc avec EXCEPTION WHEN invalid_cursor_name. Le premier OPEN reste en dehors du sous-bloc, car un bloc muni d'une section EXCEPTION s'exécute dans une sous-transaction qui est annulée en cas d'erreur : un OPEN placé à l'intérieur aurait été défait au moment de l'interception. Le bloc affiche alors « Curseur déjà ouvert (SQLSTATE 42P03) » puis « Curseur déjà fermé (SQLSTATE 34000) », ce qui donne les codes que PostgreSQL associe à ces deux erreurs.

```

DO $$
DECLARE
  n employees.nom%TYPE;
  s employees.salaire%TYPE;
  c CURSOR FOR SELECT nom, salaire FROM employees WHERE dept_id = 30;
BEGIN
  OPEN c;
  BEGIN
    OPEN c;
    EXCEPTION
      WHEN duplicate_cursor THEN
        RAISE NOTICE 'Curseur déjà ouvert (SQLSTATE %)', SQLSTATE;
    END;
    CLOSE c;
  BEGIN
    CLOSE c;
    EXCEPTION
      WHEN invalid_cursor_name THEN
        RAISE NOTICE 'Curseur déjà fermé (SQLSTATE %)', SQLSTATE;
    END;
  END;
$$;

```

Statistics 1 | Statistics 2 | Statistics 3 | Statistics 4 | Sortie X

Enter a part of a message to search for here

Nom : Garcia
 Curseur déjà ouvert (SQLSTATE 42P03)
 Curseur déjà fermé (SQLSTATE 34000)

Interception des erreurs duplicate_cursor et invalid_cursor_name

Exercice 4 : Parcourir un curseur avec une boucle

J'ai écrit quatre versions du parcours des employés du département 30, qui contient cinq personnes : Garcia, Roux, Fournier, Girard et Bonnet. La première ouvre le curseur, boucle avec LOOP, fait le FETCH au début de chaque tour et sort avec EXIT WHEN NOT FOUND juste après, avant d'afficher quoi que ce soit, puis ferme le curseur.

```

DO $$
DECLARE
  nom_employees employees.nom%TYPE;
  curs1 CURSOR FOR SELECT nom FROM employees WHERE dept_id = 30;
BEGIN
  OPEN curs1;
  LOOP
    FETCH NEXT FROM curs1 INTO nom_employees;
    EXIT WHEN NOT FOUND;
    RAISE NOTICE 'Employé : %', nom_employees;
  END LOOP;
  CLOSE curs1;
END;
$$;

```

Statistics 1 | Statistics 2 | Statistics 3 | Statistics 4 | Statistic

Enter a part of a message to search for here

Employé : Garcia
 Employé : Roux
 Employé : Fournier
 Employé : Girard
 Employé : Bonnet

Parcours avec LOOP et EXIT WHEN NOT FOUND

La deuxième version utilise WHILE FOUND LOOP. Elle demande un FETCH avant la boucle, pour que FOUND ait une valeur au premier test, et un second FETCH à la fin de chaque tour pour passer à la ligne suivante. Elle affiche les mêmes cinq employés.

```

DO $$
DECLARE
    nom_employees employees.nom%TYPE;
    curs1 CURSOR FOR SELECT nom FROM employees WHERE dept_id = 30;
BEGIN
    OPEN curs1;
    FETCH NEXT FROM curs1 INTO nom_employees;
    while found loop
        RAISE NOTICE 'Employé : %', nom_employees;
        FETCH NEXT FROM curs1 INTO nom_employees;
    END LOOP;
    CLOSE curs1;
END;
$$;

```

Statistics 1 | Statistics 2 | Résultats 3 | Statistics 4 | Statis

Enter a part of a message to search for here

Employé : Garcia
 Employé : Roux
 Employé : Fournier
 Employé : Girard
 Employé : Bonnet

Parcours avec WHILE FOUND

La troisième version utilise la boucle FOR rec IN curs1 LOOP. C'est la plus courte : PostgreSQL déclare lui-même la variable rec, ouvre le curseur, fait les FETCH, sort quand il n'y a plus de ligne et ferme le curseur, donc je n'écris ni OPEN, ni FETCH, ni CLOSE. J'accède au nom avec rec.nom.

```

DO $$
DECLARE
    curs1 CURSOR FOR SELECT nom FROM employees WHERE dept_id = 30;
BEGIN
    FOR rec IN curs1 LOOP
        RAISE NOTICE 'Employé : %', rec.nom;
    END LOOP;
END;
$$;

```

Résultats 1 | Statistics 2 | Statistics 3 | Sortie ×

Enter a part of a message to search for here

Employé : Garcia
 Employé : Roux
 Employé : Fournier
 Employé : Girard
 Employé : Bonnet

Parcours avec la boucle FOR

Dans la quatrième version, j'ai placé EXIT WHEN NOT FOUND après le RAISE NOTICE au lieu de le mettre juste après le FETCH. La sortie affiche alors une sixième ligne « Employé : <NULL> ». Lorsque le FETCH ne trouve plus de ligne, il met la variable à NULL, et comme l'affichage est exécuté avant le test de sortie, il est fait une fois de trop. Le test de FOUND doit donc suivre immédiatement le FETCH.

```

DO $$
DECLARE
  nom_employees employees.nom%TYPE;
  curs1 CURSOR FOR SELECT nom FROM employees WHERE dept_id = 30;
BEGIN
  OPEN curs1;
  LOOP
    FETCH NEXT FROM curs1 INTO nom_employees;

    RAISE NOTICE 'Employé : %', nom_employees;
  EXIT WHEN NOT FOUND;
  END LOOP;
  CLOSE curs1;
END;
$$;

```

Résultats 1 Statistics 2 Statistics 3 Statistics 4 Statistics 5

Enter a part of a message to search for here

Employé : Garcia
 Employé : Roux
 Employé : Pournier
 Employé : Girard
 Employé : Bonnet
 Employé : <NULL>

EXIT WHEN NOT FOUND placé après l'affichage : une ligne NULL en trop

Exercice 5 : Curseur paramétré et boucle imbriquée

J'ai déclaré un curseur paramétré `c_emp` avec deux paramètres, `p_dept` et `p_seuil`, qui renvoie le nom et le salaire des employés du département donné dont le salaire dépasse le seuil. Un même curseur peut ainsi servir pour plusieurs départements et plusieurs seuils, sans recopier la requête. Je l'ai appelé d'abord avec (30, 3000), qui renvoie Garcia à 4100 et Roux à 3600, puis avec (10, 2000), qui renvoie Martin à 3200, Durand à 2800 et Petit à 2800.

```
FOR rec IN c_emp(30, 3000) LOOP
```

```

DO $$
DECLARE
  c_emp CURSOR (p_dept numeric, p_seuil numeric) FOR
  SELECT nom, salaire FROM employees WHERE dept_id = p_dept AND salaire > p_seuil;
BEGIN
  RAISE NOTICE '--- (30, 3000)';
  FOR rec IN c_emp(30, 3000) LOOP
    RAISE NOTICE '% : %', rec.nom, rec.salaire;
  END LOOP;

  RAISE NOTICE '--- (10, 2000)';
  FOR rec IN c_emp(10, 2000) LOOP
    RAISE NOTICE '% : %', rec.nom, rec.salaire;
  END LOOP;
END;
$$;

```

Résultats 1 Statistics 2 Statistics 3 Statistics 4 Statistics 5 Résultats

Enter a part of a message to search for here

--- (30, 3000)
 Garcia : 4100.00
 Roux : 3600.00
 --- (10, 2000)
 Martin : 3200.00
 Durand : 2800.00
 Petit : 2800.00

Curseur paramétré appelé avec (30, 3000) puis (10, 2000)

Ensuite, j'ai ajouté une boucle externe qui parcourt tous les départements avec `FOR dep IN SELECT ... FROM departements`. J'ai déclaré la variable `dep` de type record, pour qu'elle reçoive les colonnes `dept_id` et `dept_nom` de chaque ligne du `SELECT`. À chaque tour, j'ouvre le curseur avec `dep.dept_id` et un seuil fixé à 2500. Les départements 10 et 30 affichent leurs employés, les départements 20 et 40 n'affichent aucune ligne.

```

DO $$
DECLARE
  dep record;
  c_emp CURSOR (p_dept numeric, p_seuil numeric) FOR
    SELECT nom, salaire FROM employees WHERE dept_id = p_dept AND salaire > p_seuil;
BEGIN
  FOR dep IN SELECT dept_id, dept_nom FROM departements ORDER BY dept_id LOOP
    RAISE NOTICE 'Département % (%)', dep.dept_id, dep.dept_nom;
    FOR rec IN c_emp(dep.dept_id, 2500) LOOP
      RAISE NOTICE ' % : %', rec.nom, rec.salaire;
    END LOOP;
  END LOOP;
END;
$$;

```

Statistics 1 | Sortie X

Enter a part of a message to search for here

```

Département 10 (Compta)
Martin : 3200.00
Durand : 2800.00
Petit : 2800.00
Département 20 (RH)
Département 30 (IT)
Garcia : 4100.00
Roux : 3600.00
Fournier : 3000.00
Département 40 (Archives)

```

Boucle sur les départements avec un seuil de 2500

Pour signaler les départements sans employé au-dessus du seuil, j'ai ajouté IF NOT FOUND THEN ... END IF juste après la boucle interne. Je l'avais d'abord mis à l'intérieur de la boucle, où il ne pouvait jamais se déclencher, puisque la boucle ne s'exécute que lorsqu'il y a une ligne. Après la boucle, FOUND vaut vrai si la boucle a fait au moins un tour et faux sinon. Le message « aucun employé » apparaît alors pour les départements 20 (RH) et 40 (Archives).

```

DO $$
DECLARE
  dep record;
  c_emp CURSOR (p_dept numeric, p_seuil numeric) FOR
    SELECT nom, salaire FROM employees WHERE dept_id = p_dept AND salaire > p_seuil;
BEGIN
  FOR dep IN SELECT dept_id, dept_nom FROM departements ORDER BY dept_id LOOP
    RAISE NOTICE 'Département % (%)', dep.dept_id, dep.dept_nom;
    FOR rec IN c_emp(dep.dept_id, 2500) LOOP
      RAISE NOTICE ' % : %', rec.nom, rec.salaire;
    END LOOP;

    IF NOT FOUND THEN
      RAISE NOTICE ' aucun employé';
    END IF;
  END LOOP;
END;
$$;

```

Statistics 1 | Statistics 2 | Sortie X

Enter a part of a message to search for here

```

Département 10 (Compta)
Martin : 3200.00
Durand : 2800.00
Petit : 2800.00
Département 20 (RH)
aucun employé
Département 30 (IT)
Garcia : 4100.00
Roux : 3600.00
Fournier : 3000.00
Département 40 (Archives)
aucun employé

```

Message « aucun employé » pour les départements vides

Enfin, j'ai calculé la moyenne des salaires affichés pour chaque département. Pour cela, un total et un compteur sont remis à zéro au début de chaque tour de la boucle externe, puis mis à jour dans la boucle interne. La moyenne est calculée avec $\text{round}(\text{total} / \text{nb}, 2)$ uniquement lorsque nb est supérieur à 0, sinon j'affiche « sans objet ». Sans ce test, la division par zéro des départements vides ferait échouer le bloc. J'obtiens 2933.33 pour le département 10 et 3566.67 pour le département 30.

```

DO $$
DECLARE
    dep record;
    total numeric;
    nb integer;
    c_emp CURSOR (p_dept numeric, p_seuil numeric) FOR
        SELECT nom, salaire FROM employees WHERE dept_id = p_dept AND salaire > p_seuil;
BEGIN
    FOR dep IN SELECT dept_id, dept_nom FROM departements ORDER BY dept_id LOOP
        RAISE NOTICE 'Département % (%)', dep.dept_id, dep.dept_nom;
        total := 0;
        nb := 0;

        FOR rec IN c_emp(dep.dept_id, 2500) LOOP
            RAISE NOTICE ' % : %', rec.nom, rec.salaire;
            total := total + rec.salaire;
            nb := nb + 1;
        END LOOP;

        IF NOT FOUND THEN
            RAISE NOTICE ' aucun employé';
        END IF;

        IF nb > 0 THEN
            RAISE NOTICE ' moyenne : %', round(total / nb, 2);
        ELSE
            RAISE NOTICE ' moyenne : sans objet';
        END IF;
    END LOOP;
END;
$$;

```

Statistics 1 | Statistics 2 | Sortie X

Enter a part of a message to search for here

```

Département 10 (Compta)
Martin : 3200.00
Durand : 2800.00
Petit : 2800.00
moyenne : 2933.33
Département 20 (RH)
aucun employé
moyenne : sans objet
Département 30 (IT)
Garcia : 4100.00
Roux : 3600.00
Fournier : 3000.00
moyenne : 3566.67
Département 40 (Archives)
aucun employé
moyenne : sans objet

```

Moyenne par département, « sans objet » pour les départements vides

Exercice 6 : Verrouiller et modifier avec FOR UPDATE

Cet exercice met en jeu plusieurs sessions en même temps. Dans la session 1, j'ai déclaré un curseur avec FOR UPDATE sur les employés du département 10 dont la prime est NULL, puis je l'ai parcouru avec UPDATE ... WHERE CURRENT OF c, qui modifie exactement la ligne que le curseur vient de lire. J'ai ajouté PERFORM pg_sleep(60), l'équivalent de DBMS_SESSION.SLEEP d'Oracle, pour que la transaction reste ouverte et que les verrous restent posés. Cette session était en mode de validation manuelle dans DBeaver, donc aucun UPDATE n'était validé avant mon COMMIT. Le panneau Statistics affiche une durée de 1m 0s.

```
UPDATE employees SET prime = rec.salaire * 0.05 WHERE CURRENT OF c;
```

```

DO $$
DECLARE
    c CURSOR FOR SELECT emp_id, salaire FROM employees
        WHERE dept_id = 10 AND prime IS NULL FOR UPDATE;
BEGIN
    FOR rec IN c LOOP
        UPDATE employees SET prime = rec.salaire * 0.05 WHERE CURRENT OF c;
    END LOOP;
    PERFORM pg_sleep(60);
END;
$$;

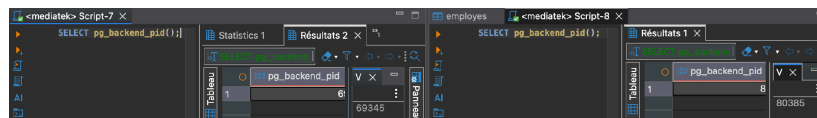
```

Statistics 1 X

Name	Value
Updated Rows	0
Execute time	1m 0s
Start time	Fri Oct 09 13:55:15 CEST 2026
Finish time	Fri Oct 09 13:56:15 CEST 2026
Query	DO \$\$ DECLARE c CURSOR FOR SELECT emp_id, salaire FROM employees WHERE dept_id = 10 AND prime IS NULL FOR UPDATE; BEGIN FOR rec IN c LOOP UPDATE employees SET prime = rec.salaire * 0.05 WHERE CURRENT OF c; END LOOP; PERFORM pg_sleep(60); END; \$\$

Session 1 : curseur FOR UPDATE et pg_sleep(60)

Pour jouer le rôle d'autres utilisateurs, j'ai ouvert deux autres éditeurs SQL, chacun avec sa propre connexion. J'ai vérifié qu'il s'agissait bien de sessions différentes avec `SELECT pg_backend_pid()`, qui renvoie 69345 dans l'un et 80385 dans l'autre.



Identifiants de processus des deux sessions avec `pg_backend_pid()`

Dans la session 2, j'ai lancé un `UPDATE` qui ajoute 1 au salaire de Durand, une ligne verrouillée par la session 1. Il est resté en attente et n'a abouti que lorsque j'ai validé la session 1 avec `COMMIT` : le panneau Statistics indique une durée de 4m 25s, de 14:10:04 à 14:14:30, avec 1 ligne modifiée.

Name	Value
Updated Rows	1
Execute time	4m 25s
Start time	Fri Oct 09 14:10:04 CEST 2026
Finish time	Fri Oct 09 14:14:30 CEST 2026
Query	UPDATE employees SET salaire = salaire + 1 WHERE

Session 2 : `UPDATE` resté en attente pendant 4m 25s

Dans la session 3, j'ai lu les employés du département 10 avant et après le `COMMIT`. Avant le `COMMIT`, les primes de Durand et de Leroy sont encore à `NULL` : les modifications d'une transaction non validée ne sont pas visibles par les autres sessions. Après le `COMMIT`, Durand a un salaire de 2801 et une prime de 140, Leroy une prime de 95. La prime de Durand vaut 5 % de 2800, c'est-à-dire son salaire avant l'augmentation de la session 2, ce qui confirme que la session 2 n'a pu modifier cette ligne qu'une fois le verrou de la session 1 libéré.

employees 1 X			
SELECT nom, salaire, prime FF Entrez une expression SQL			
			123 prime
1	Martin	3 200	500
2	Durand	2 800	[NULL]
3	Petit	2 800	300
4	Leroy	1 900	[NULL]

Session 3 avant le `COMMIT` : primes de Durand et Leroy à `NULL`

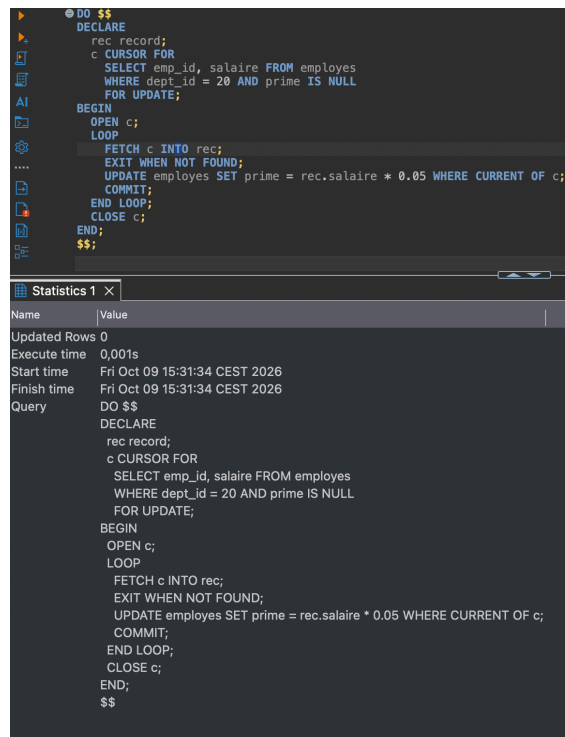
SELECT nom, salaire, prime FF Entrez une expression SQL			
	AZ nom	123 salaire	123 prime
1	Martin	3 200	500
2	Durand	2 801	140
3	Petit	2 800	300
4	Leroy	1 900	95

Session 3 après le `COMMIT` : primes renseignées et salaire de Durand à 2801

Pour la variante `NOWAIT`, j'ai refait le même curseur avec `FOR UPDATE NOWAIT` pendant qu'une autre session retenait les lignes du département 10 avec un `SELECT ... FOR UPDATE` non validé. Au lieu d'attendre comme dans la session 2, la session 1 échoue tout de suite. J'ai placé le parcours dans un

sous-bloc avec EXCEPTION WHEN lock_not_available, qui affiche « Lignes verrouillées par une autre session » avec le SQLSTATE 55P03. J'ai ensuite fait un ROLLBACK dans l'autre session pour libérer le verrou, et DBeaver a confirmé par le message « Transaction has been rolled back ».

Enfin, j'ai placé un COMMIT dans la boucle du curseur FOR UPDATE, juste après le UPDATE. Sur le département 20, où un seul employé (Simon) a une prime NULL, le bloc se termine sans erreur et Simon reçoit une prime de 105, soit 5 % de 2100. Moreau et Laurent avaient déjà une prime et n'ont pas été modifiés.



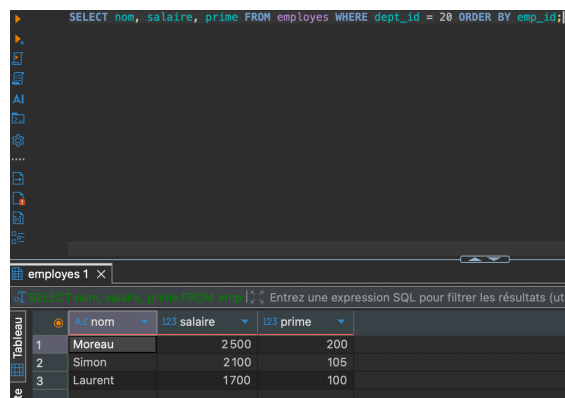
```

DO $$
DECLARE
    rec record;
    c CURSOR FOR
        SELECT emp_id, salaire FROM employees
        WHERE dept_id = 20 AND prime IS NULL
        FOR UPDATE;
BEGIN
    OPEN c;
    LOOP
        FETCH c INTO rec;
        EXIT WHEN NOT FOUND;
        UPDATE employees SET prime = rec.salaire * 0.05 WHERE CURRENT OF c;
        COMMIT;
    END LOOP;
    CLOSE c;
END;
$$

```

Name	Value
Updated Rows	0
Execute time	0,001s
Start time	Fri Oct 09 15:31:34 CEST 2026
Finish time	Fri Oct 09 15:31:34 CEST 2026
Query	DO \$\$ DECLARE rec record; c CURSOR FOR SELECT emp_id, salaire FROM employees WHERE dept_id = 20 AND prime IS NULL FOR UPDATE; BEGIN OPEN c; LOOP FETCH c INTO rec; EXIT WHEN NOT FOUND; UPDATE employees SET prime = rec.salaire * 0.05 WHERE CURRENT OF c; COMMIT; END LOOP; CLOSE c; END; \$\$

Bloc avec COMMIT dans la boucle, exécuté sur le département 20



```

SELECT nom, salaire, prime FROM employees WHERE dept_id = 20 ORDER BY emp_id;

```

	123 nom	123 salaire	123 prime
1	Moreau	2500	200
2	Simon	2100	105
3	Laurent	1700	100

Primes du département 20 après l'exécution

Sur le département 10, où Durand et Leroy ont une prime NULL, le même bloc s'arrête avec l'erreur 34000 « cursor "<unnamed portal 1>" does not exist » à la ligne du FETCH. J'en déduis que le COMMIT termine la transaction qui portait le verrou du curseur FOR UPDATE, ce qui supprime le curseur, si bien que le FETCH suivant ne le trouve plus. Je n'ai pas approfondi pourquoi le bloc passe sans erreur lorsqu'il n'y a qu'une seule ligne à traiter.

```

DO $$
DECLARE
    rec emp%ROWTYPE;
    c CURSOR FOR
        SELECT emp_id, salaire FROM employes
        WHERE dept_id = 10 AND prime IS NULL;
BEGIN
    OPEN c;
    LOOP
        FETCH c INTO rec;
        EXIT WHEN NOT FOUND;
        UPDATE employes SET prime = rec.salaire * 0.05 WHERE CURRENT OF c;
    END LOOP;
    COMMIT;
    CLOSE c;
END;
$$;

```

SQL Error [34000]: ERROR: cursor "unnamed portal 1>" does not exist
 Où : PL/pgSQL function inline_code_block line 11 at FETCH

Même bloc sur le département 10 : erreur 34000 au FETCH

Cet exercice m'a montré à quoi sert FOR UPDATE : il verrouille les lignes lues par le curseur jusqu'au COMMIT ou au ROLLBACK, ce qui empêche une autre session de les modifier entre le moment où je les lis et celui où je les mets à jour. WHERE CURRENT OF est plus précis qu'un WHERE emp_id = ... car il agit exactement sur la ligne courante du curseur, sans devoir la retrouver par sa clé et sans risque de modifier une autre ligne.

Exercice 7 : Mini-projet, rapport de paie par département

Pour le mini-projet, j'ai écrit un bloc DO qui produit un rapport de paie par département. Il utilise un curseur paramétré c_emp(p_dept), qui renvoie le nom, le salaire et la prime des employés d'un département triés par emp_id, appelé dans une boucle FOR imbriquée dans une boucle FOR sur la table departements. Pour chaque employé, il affiche le salaire et la prime, en remplaçant une prime NULL par 0 avec COALESCE(rec.prime, 0). Les noms sont alignés avec rpad(rec.nom, 12). Le total d'un département additionne salaire et prime, ce que j'ai indiqué dans le titre du rapport, alors que le salaire le plus haut est calculé sur le salaire seul avec GREATEST.

```

total_dept := total_dept + rec.salaire + COALESCE(rec.prime, 0);
max_dept := GREATEST(max_dept, rec.salaire);

```

Un département sans employé est détecté avec IF NOT FOUND après la boucle interne : le rapport affiche alors « Aucun employé dans ce département » à la place du total et du maximum, qui seraient trompeurs. À la fin de chaque département, j'ajoute son total à total_general, affiché en dernière ligne. La section EXCEPTION sert à deux choses. Au début du bloc, je compte les départements et je déclenche un RAISE EXCEPTION si la table est vide, car il n'y aurait rien à rapporter. Ce cas est intercepté par WHEN raise_exception, qui affiche « Rapport annulé » avec le message d'origine. Un WHEN OTHERS affiche le SQLSTATE et le SQLERRM de toute autre erreur, afin qu'elle ne passe pas inaperçue.

```

DO $$
DECLARE
    dep record;
    rec record;
    nb_dep integer;
    total_dept numeric := 0;
    max_dept numeric := 0;
    total_general numeric := 0;

    c_emp CURSOR (p_dept numeric) FOR
        SELECT nom, salaire, prime
        FROM employees
        WHERE dept_id = p_dept
        ORDER BY emp_id;
BEGIN
    SELECT count(*) INTO nb_dep FROM departements;
    IF nb_dep = 0 THEN
        RAISE EXCEPTION 'Aucun département à traiter';
    END IF;

    RAISE NOTICE '==== RAPPORT DE PAIE (total = salaire + prime) =====';

    FOR dep IN SELECT dept_id, dept_nom FROM departements ORDER BY dept_id LOOP
        total_dept := 0;
        max_dept := 0;
        RAISE NOTICE '--- Département % (%) ---', dep.dept_id, dep.dept_nom;

        FOR rec IN c_emp(dep.dept_id) LOOP
            RAISE NOTICE '% : salaire %, prime %',
                rpad(rec.nom, 12), rec.salaire, COALESCE(rec.prime, 0);
            total_dept := total_dept + rec.salaire + COALESCE(rec.prime, 0);
            max_dept := GREATEST(max_dept, rec.salaire);
        END LOOP;

        IF NOT FOUND THEN
            RAISE NOTICE 'Aucun employé dans ce département';
        ELSE
            RAISE NOTICE 'Total du département : %', total_dept;
            RAISE NOTICE 'Salaire le plus haut : %', max_dept;
        END IF;

        total_general := total_general + total_dept;
    END LOOP;

    RAISE NOTICE '==== TOTAL GÉNÉRAL : % =====', total_general;

    EXCEPTION
    WHEN raise_exception THEN
        RAISE NOTICE 'Rapport annulé : %', SQLERRM;
    WHEN OTHERS THEN
        RAISE NOTICE 'Erreur imprévue (SQLSTATE %) : %', SQLSTATE, SQLERRM;
END;
$$;

```

Bloc du rapport de paie avec curseur paramétré et section EXCEPTION

Mes premières versions avaient deux défauts. Les lignes qui cumulent le total et le maximum étaient écrites après le END LOOP de la boucle interne, où rec ne contient plus que le dernier employé lu, et n'est même pas assigné pour un département vide. Je n'avais pas non plus de section EXCEPTION correcte : elle ne peut pas se trouver dans une boucle et doit être écrite une seule fois, juste avant le END final du bloc. J'ai déplacé les cumuls dans la boucle interne et placé l'exception en fin de bloc.

Le rapport affiche quatre départements. Le département 10 (Compta) totalise 11736.05 avec un salaire maximum de 3200, le département 20 (RH) 6705 avec 2500, le département 30 (IT) 16000 avec 4100, et le département 40 (Archives) affiche le message de département vide. Le total général de 34441.05 correspond bien à la somme des trois totaux de département, 11736.05 + 6705 + 16000.

```

===== RAPPORT DE PAIE (total = salaire + prime) =====
--- Département 10 (Compta) ---
Martin      : salaire 3200.00, prime 500.00
Durand      : salaire 2801.00, prime 140.05
Petit       : salaire 2800.00, prime 300.00
Leroy       : salaire 1900.00, prime 95.00
Total du département : 11736.05
Salaire le plus haut : 3200.00
--- Département 20 (RH) ---
Moreau      : salaire 2500.00, prime 200.00
Simon       : salaire 2100.00, prime 105.00
Laurent     : salaire 1700.00, prime 100.00
Total du département : 6705.00
Salaire le plus haut : 2500.00
--- Département 30 (IT) ---
Garcia      : salaire 4100.00, prime 900.00
Roux        : salaire 3600.00, prime 0
Fournier    : salaire 3000.00, prime 400.00
Girard      : salaire 2200.00, prime 0
Bonnet      : salaire 1800.00, prime 0
Total du département : 16000.00
Salaire le plus haut : 4100.00
--- Département 40 (Archives) ---
Aucun employé dans ce département
===== TOTAL GÉNÉRAL : 34441.05 =====

```

Rapport de paie par département et total général

Compte rendu final

Dans quel cas utiliser un curseur explicite plutôt qu'un SELECT INTO ? J'utilise un curseur explicite lorsque la requête peut renvoyer plusieurs lignes que je dois traiter une par une, par exemple pour afficher les employés d'un département ou calculer une moyenne (exercices 4, 5 et 7). SELECT INTO ne convient que pour une seule ligne : sans STRICT, il ignore les lignes en trop et ne dit rien s'il n'y en a aucune, et avec STRICT il lève une erreur dans les deux cas (exercice 1). Le curseur est aussi nécessaire quand la requête doit être paramétrée et réutilisée avec plusieurs valeurs (exercice 5), ou quand je veux modifier la ligne en cours avec FOR UPDATE et WHERE CURRENT OF (exercice 6). Pour une seule ligne dont je suis sûr, SELECT INTO reste plus court et plus lisible.

Quelle erreur sur les curseurs m'a le plus surpris ? C'est celle de l'exercice 6, avec le COMMIT placé dans la boucle du curseur FOR UPDATE. Le même bloc se termine sans erreur sur le département 20 mais s'arrête sur le département 10 avec l'erreur « cursor does not exist ». Je ne m'attendais pas à ce qu'une validation de transaction supprime le curseur, ni à ce que le résultat dépende du nombre de lignes à traiter. J'en ai retenu qu'avec un curseur FOR UPDATE, la transaction et le curseur sont liés et qu'on ne peut pas valider au milieu de la boucle.

Une chose qui n'a pas marché du premier coup : le rapport de paie de l'exercice 7. Mes cumuls du total et du salaire maximum étaient écrits après la boucle interne, donc ils ne tenaient compte que du dernier employé de chaque département et auraient provoqué une erreur sur un département vide, et j'avais placé la section EXCEPTION dans la boucle. Je les ai résolus en déplaçant les cumuls dans la boucle interne, qui parcourt chaque employé, et en écrivant une seule fois la section EXCEPTION juste avant le END du bloc. Les totaux affichés sont maintenant cohérents avec les valeurs de chaque département.

Conclusion

Ce TD m'a permis de comprendre le cycle de vie d'un curseur (déclaration, OPEN, FETCH, CLOSE) et ses trois façons d'être parcouru : LOOP avec EXIT WHEN NOT FOUND, WHILE FOUND, et la boucle FOR qui fait tout automatiquement. J'ai aussi vu comment adapter à PostgreSQL les notions d'Oracle qui n'existent pas, comme les attributs de curseur, en utilisant pg_cursors, FOUND et un compteur. Les erreurs de l'exercice 3 m'ont appris à reconnaître leurs codes SQLSTATE et à les intercepter avec une section EXCEPTION.

L'exercice 6 m'a fait comprendre les verrous avec plusieurs sessions : FOR UPDATE bloque une autre session jusqu'au COMMIT, NOWAIT la fait échouer immédiatement, et un COMMIT dans la boucle détruit le curseur. L'exercice 7 a rassemblé curseur paramétré, boucles imbriquées, traitement des NULL, cas particulier du département vide et gestion d'erreurs dans un seul rapport. J'ai rencontré plusieurs erreurs sur la route (bloc sans DO, EXIT mal placé, cumuls hors de la boucle), mais elles m'ont aidé à bien comprendre où se placent les instructions dans une boucle de curseur.